

1. DFA Simulator

Code

```
def dfa(s):
    state = "q0"
    path = [state]

    for ch in s:

        if state == "q0":
            if ch == "a":
                state = "q1"
            else:
                state = "q0"

        elif state == "q1":
            if ch == "a":
                state = "q1"
            else:
                state = "q2"

        elif state == "q2":
            if ch == "a":
                state = "q1"
            else:
                state = "q0"

        path.append(state)

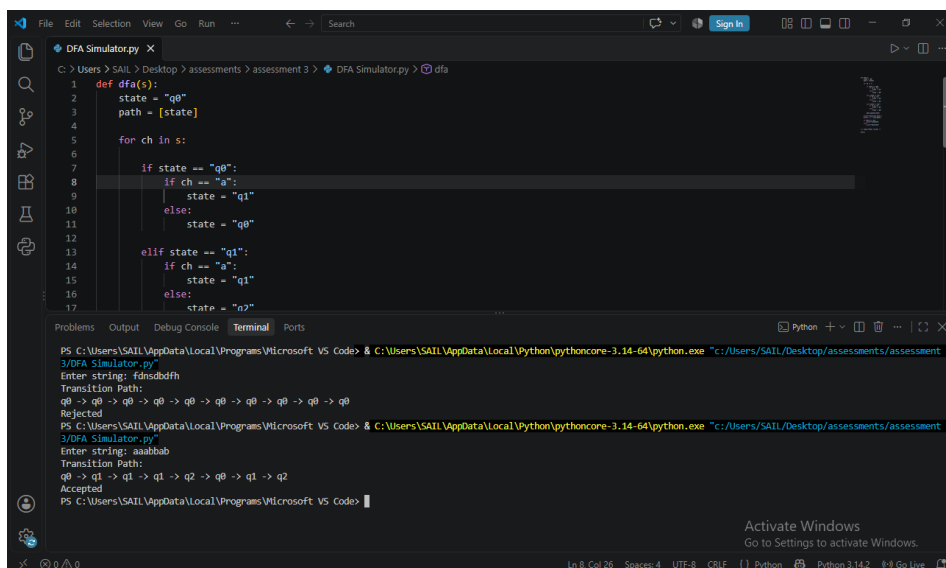
    print("Transition Path:")
    print(" -> ".join(path))

    if state == "q2":
        print("Accepted")
    else:
        print("Rejected")
```

```
s = input("Enter string: ")
```

```
dfa(s)
```

Output



The screenshot shows a VS Code editor window with a file named 'DFA Simulator.py'. The code is a Python function 'dfa(s)' that simulates a DFA. It starts at state 'q0' and transitions based on input characters 'a'. The states are 'q0', 'q1', and 'q2'. The function prints the transition path and whether the string is accepted or rejected. The terminal output shows two test cases: 'fdnsdbdfh' which is rejected, and 'aaabab' which is accepted. The terminal output for 'aaabab' shows the transition path: 'q0 -> q1 -> q1 -> q1 -> q2 -> q0 -> q1 -> q2'.

```
PS C:\Users\SAIL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\SAIL\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/SAIL/Desktop/assessments/assessment
3/DFA Simulator.py"
Enter string: fdnsdbdfh
Transition Path:
q0 -> q0 -> q0 -> q0 -> q0 -> q0 -> q0 -> q0 -> q0
Rejected
PS C:\Users\SAIL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\SAIL\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/SAIL/Desktop/assessments/assessment
3/DFA Simulator.py"
Enter string: aaabab
Transition Path:
q0 -> q1 -> q1 -> q1 -> q2 -> q0 -> q1 -> q2
Accepted
PS C:\Users\SAIL\AppData\Local\Programs\Microsoft VS Code>
```

2. Intelligent Email and Password Validator

Code

```
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

# Email validation
email_pattern = r"^[A-Za-z][A-Za-z0-9-_.]*@[A-Za-z]+\.(com|org|edu|net|in)$"

# Password validation
password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$"

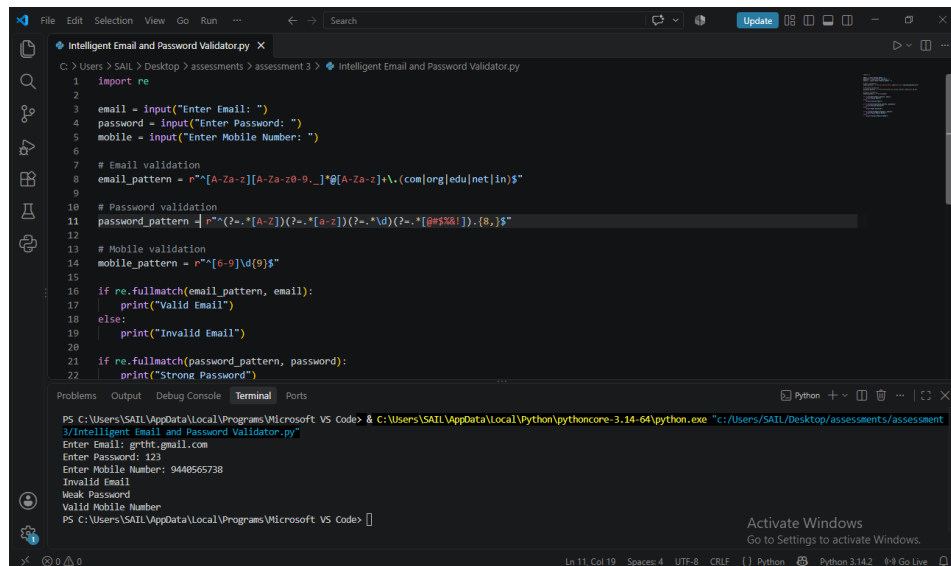
# Mobile validation
mobile_pattern = r"^[6-9]\d{9}$"

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Output



The screenshot shows a Visual Studio Code editor window with a file named "Intelligent Email and Password Validator.py". The code is as follows:

```
1 import re
2
3 email = input("Enter Email: ")
4 password = input("Enter Password: ")
5 mobile = input("Enter Mobile Number: ")
6
7 # Email validation
8 email_pattern = r"^[A-Za-z][A-Za-z0-9-_.]*@[A-Za-z]+\.(com|org|edu|net|in)$"
9
10 # Password validation
11 password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$"
12
13 # Mobile validation
14 mobile_pattern = r"^[6-9]\d{9}$"
15
16 if re.fullmatch(email_pattern, email):
17     print("Valid Email")
18 else:
19     print("Invalid Email")
20
21 if re.fullmatch(password_pattern, password):
22     print("Strong Password")
```

The terminal output at the bottom shows the execution results:

```
PS C:\Users\SAIL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\SAIL\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/SAIL/Desktop/assessments/assessment
3/Intelligent Email and Password Validator.py"
Enter Email: grt@t.gmail.com
Enter Password: 123
Enter Mobile Number: 9448565738
Invalid Email
Weak Password
Valid Mobile Number
PS C:\Users\SAIL\AppData\Local\Programs\Microsoft VS Code>
```

The status bar at the bottom indicates the file is at line 11, column 19, using UTF-8 encoding with CRLF line endings. The Python interpreter is set to Python 3.14.2.

3. Smart Pattern Matching Engine

Code

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

print("1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")

choice = input("Enter choice: ")

if choice == "1":
    result = re.findall(
        r"\b\d{2}/\d{2}/\d{4}\b",
        text
    )

elif choice == "2":
    result = re.findall(
        r"\b[6-9]\d{9}\b",
        text
    )

elif choice == "3":
    result = re.findall(
        r"#\w+",
        text
    )

elif choice == "4":
    result = re.findall(
        r"@ \w+",
        text
    )

elif choice == "5":
    word = input("Enter prefix: ")

    result = re.findall(
        r"\b" + re.escape(word) + r"\w*",
        text,
        re.I
    )

elif choice == "6":
    word = input("Enter suffix: ")

    result = re.findall(
        r"\w*" + re.escape(word) + r"\b",
        text,
        re.I
    )
```

else:

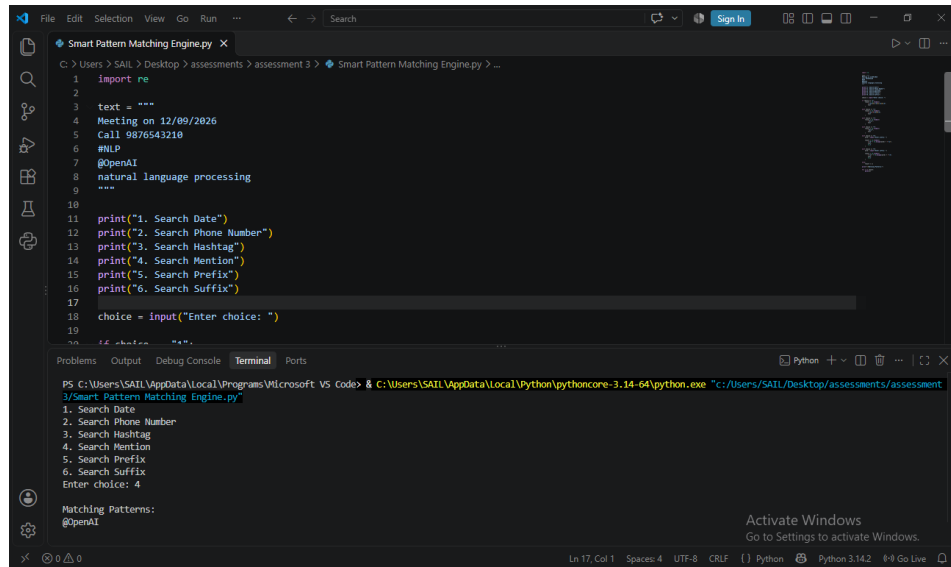
```
    result = []
```

```
print("\nMatching Patterns:")
```

```
for x in result:
```

```
    print(x)
```

Output



The screenshot shows a Visual Studio Code editor window with a Python file named "Smart Pattern Matching Engine.py". The code defines a text string and a list of patterns, then prints a menu of search options. The terminal output shows the program running and the user selecting option 4 (Search Prefix).

```
1 import re
2
3 text = """
4 Meeting on 12/09/2026
5 Call 9876543210
6
7 #NLP
8 @OpenAI
9 natural language processing
10
11 print("1. Search Date")
12 print("2. Search Phone Number")
13 print("3. Search Hashtag")
14 print("4. Search Mention")
15 print("5. Search Prefix")
16 print("6. Search Suffix")
17
18 choice = input("Enter choice: ")
19
20
```

Terminal Output:

```
PS C:\Users\SAIL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\SAIL\AppData\Local\Python\pythoncore-3.14-64\python.exe "C:\Users\SAIL\Desktop\assessments\assessment
3\Smart Pattern Matching Engine.py"
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
Enter choice: 4

Matching Patterns:
@OpenAI
```